DOC-SMCTA-GOV-003 · VERSIÓN 2.1.0

Especificación de Casos de Uso Críticos (Formato Cockburn Fully Dressed)

Contratos Operativos Paso a Paso, Invariantes Pre/Post y Extensiones de Error para Custodia, P2P y TPV

Proyecto / Sistema	**Especificación Formal de Requisitos bajo ISO/IEC/IEEE 29148 y Alistair Cockburn**
Fase / Área Normativa	**06 — Gobernanza de Ingeniería y Blindaje Técnico Oficial**
Autoría y Gobernanza	**Equipo de Ingeniería de Software — Proyecto SMCTA**
Fecha de Emisión	**Noviembre 2026**

Resumen

Mientras las historias de usuario ágiles capturan intenciones generales de producto, los flujos que movilizan dinero en custodia, transfieren titularidad bajo concurrencia o liberan fondos en terminales físicos exigen contratos deterministas sin margen de interpretación entre roles. Este documento especifica en formato **Alistair Cockburn *Fully Dressed*** los cuatro casos de uso nucleares de **SMCTA v2** (`UC-01` a `UC-04`), detallando actores, intereses de los *stakeholders*, precondiciones verificables, garantías mínimas ante fallo (*rollback*), garantías de éxito y todas las ramificaciones exactas de códigos de error HTTP implementadas en producción.

Índice General de Contenidos

Resumen
Autómata Finito de Estados del Cupón (`CouponState`)
UC-01: Emisión Primaria de Cupón y Fondeo Inicial de Custodia Escrow
Escenario Principal de Éxito (`UC-01`)
Extensiones y Flujos Alternativos (`UC-01`)
UC-02: Publicación de Oferta en Mercado Secundario P2P con Price Collar y Cuota Diaria
Escenario Principal de Éxito (`UC-02`)
Extensiones y Flujos Alternativos (`UC-02`)
UC-03: Ejecución Atómica de Compra Secundaria y Liquidación de Comisiones (Split P2P)
Escenario Principal de Éxito (`UC-03`)
Extensiones y Flujos Alternativos (`UC-03`)
UC-04: Validación Criptográfica de QR Dinámico y Liquidación de Custodia en TPV
Escenario Principal de Éxito (`UC-04`)
Extensiones y Flujos Alternativos (`UC-04`)

Autómata Finito de Estados del Cupón (`CouponState`)

Todos los casos de uso operan gobernados por el autómata determinista implementado en `packages/shared/src/stateMachine.ts` (`ALLOWED_TRANSITIONS`), cuya representación formal se ilustra a continuación:

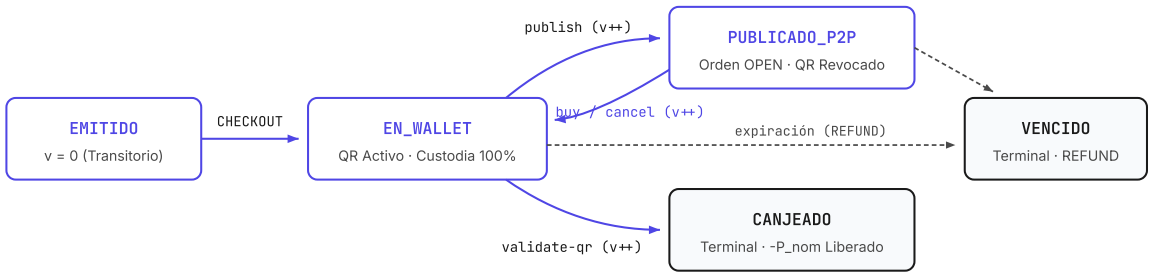


Figura 1. Autómata finito determinista de estados del cupón (`packages/shared/src/stateMachine.ts`). Cada arista con `v++` incrementa atómicamente `coupons.token_version` , revocando cualquier captura QR previa.

UC-01: Emisión Primaria de Cupón y Fondeo Inicial de Custodia Escrow

Atributo Normativo	Especificación Formal
Identificador y Endpoint	`UC-01` — `POST /api/v1/checkout/primary` (`checkoutService.primaryCheckout`)
Nivel y Alcance	Nivel Usuario (*Sea-Level*) — Caja negra transaccional sobre `@smcta/backend`
Actor Primario	Consumidor Primario autenticado bajo el contexto de un *Tenant* válido
Stakeholders e Intereses	**Consumidor:** Obtener titularidad inmediata de su cupón respaldado al 100%. **Comercio (*Tenant*):** Asegurar que el pago quede constituido sin riesgo de contracargo informal. **Auditoría:** Impedir que exista un cupón en cartera sin su asiento `CHECKOUT` correlativo.
Precondiciones	1. El encabezado `x-tenant-subdomain` o `Host` resuelve a un registro activo en `tenants` . 2. El usuario se encuentra identificado (`buyerId` / `x-user-id`). 3. `nominalPrice > 0` y `expirationDate` es una fecha futura válida en ISO-8601.
Garantía Mínima (En Fallo)	Transacción abortada (`ROLLBACK`); cero filas insertadas en `coupons` , `escrow_ledgers` ni `user_events` ; cero emisiones en `EventHub` .
Garantía de Éxito (Postcondición)	Cupón creado en estado `EN_WALLET` con `token_version = 0` ; asiento inmutable `CHECKOUT` por `+nominalPrice` registrado en `escrow_ledgers` ; notificación persistida y emitida vía SSE post- `COMMIT` .

Escenario Principal de Éxito (`UC-01`)

1. El Consumidor selecciona uno o más productos en la tienda *White-Label* (o en el carrito `CartDrawer` de *BREW/DESK*) y envía la orden de compra primaria indicando `nominalPrice`, `productName` y `expirationDate`.
2. El middleware `tenantResolver` resuelve el *Tenant* activo e inyecta su configuración y `QrSecret` en el contexto de la petición.
3. `checkoutService` valida la estructura con Zod, verifica coherencia temporal respecto al reloj inyectado (`Clock.now()`) y abre una unidad de trabajo `SERIALIZABLE`.
4. El sistema inserta el registro inicial del cupón en `coupons` y ejecuta la transición formal `EMITIDO → EN_WALLET` verificada por `assertTransition`.
5. El sistema inserta en `escrow_ledgers` el asiento `transaction_type = 'CHECKOUT'` con `amount_in_escrow = +nominalPrice`, protegido por el índice único parcial `uq_ledger_checkout_once`.
6. El sistema registra el evento `PRIMARY_PURCHASE` en `user_events` y confirma la transacción (`COMMIT`).
7. Una vez confirmado el `COMMIT`, el servicio publica el delta en `EventHub` (canal SSE `/api/v1/notifications/stream`) y responde `HTTP 201 Created` con el `CouponDTO`.

Extensiones y Flujos Alternativos (`UC-01`)

- **2a. Subdominio inexistente o no registrado:** `tenantResolver` aborta antes de tocar servicios de dominio y devuelve `HTTP 404 Not Found` (`TENANT_NOT_FOUND`).
- **3a. Importe nominal nulo, negativo o fecha malformada:** El validador rechaza la carga útil con `HTTP 422 Unprocessable Entity` (`VALIDATION_ERROR`).
- **3b. Identidad de comprador ausente en modo producción (`enableDevRoutes = false`):** Rechazo inmediato con `HTTP 401 Unauthorized` (`UNAUTHORIZED_USER`).

UC-02: Publicación de Oferta en Mercado Secundario P2P con Price Collar y Cuota Diaria

Atributo Normativo	Especificación Formal
Identificador y Endpoint	`UC-02` — `POST /api/v1/p2p/orders` (`p2pService.publishOrder`)
Actor Primario	Titular actual del cupón (*Vendedor Secundario*)

Atributo Normativo	Especificación Formal
Stakeholders e Intereses	**Vendedor:** Conocer de antemano su producido neto exacto y listar su cupón sin perder la garantía si cancela. **Comercio (*Tenant*):** Impedir sobreprecios abusivos o remates por debajo del piso autorizado y bloquear acaparadores. **Seguridad TPV:** Invalidar cualquier QR generado antes de listar el cupón.
Precondiciones	El cupón existe en el *Tenant* activo, pertenece a `sellerId` y su estado es exactamente `EN_WALLET`.
Garantía Mínima (En Fallo)	El cupón conserva estado `EN_WALLET` y su `token_version` intacto; no se inserta orden en `p2p_orders`.
Garantía de Éxito (Postcondición)	Cupón pasa a `PUBLICADO_P2P` con `token_version = token_version + 1`; se crea exactamente una orden `OPEN` en `p2p_orders` con el desglose decimal calculado.

Escenario Principal de Éxito (`UC-02`)

1. El Vendedor ajusta el precio deseado `askingPrice` en el control deslizante de su billetera (que muestra en vivo la banda permitida y su neto estimado) y confirma la publicación.
2. `p2pService` adquiere exclusión mutua sobre la clave `"coupon:" + couponId` mediante `LockManager`.
3. Dentro de una transacción `SERIALIZABLE`, el repositorio lee y bloquea la fila del cupón (`SELECT ... FOR UPDATE`), verificando pertenencia al *Tenant* y coincidencia exacta `coupon.currentOwnerId === sellerId`.
4. El sistema verifica que el cupón se encuentre en estado `EN_WALLET` y que el tiempo restante hasta `expirationDate` supere estrictamente `tenant.closureHoursBeforeEvent`.
5. El sistema invoca `validatePriceCollar(nominalPrice, askingPrice, priceFloorPct, priceCeilingPct)` confirmando que $P_{ask} \in [P_{min}, P_{max}]$.
6. El sistema contabiliza en `p2p_orders` las órdenes creadas por `sellerId` en el día UTC en curso y verifica que su cantidad sea menor que `tenant.maxDailyResalesPerUser`.
7. El sistema actualiza el cupón a `PUBLICADO_P2P` incrementando `token_version + 1` (revocando cualquier QR previo en poder del vendedor) e inserta la nueva orden `OPEN` en `p2p_orders`.
8. Tras confirmar el `COMMIT`, emite notificación SSE y responde `HTTP 201 Created` con `P2POrderDTO` y `commissionBreakdown`.

Extensiones y Flujos Alternativos (`UC-02`)

- **3a. Usuario solicitante no es el dueño actual del cupón:** Aborta con `HTTP 403 Forbidden` (`FORBIDDEN_NOT_OWNER`).
- **4a. El cupón ya está publicado, canjeado o vencido:** Aborta con `HTTP 409 Conflict` (`COUPON_NOT_IN_WALLET`).
- **4b. Ventana de cierre pre-evento alcanzada ($\Delta t \leq$ closureHoursBeforeEvent):** Aborta con `HTTP 423 Locked` (`MARKET_CLOSED_BEFORE_EVENT`).
- **5a. Precio fuera de la banda paramétrica $[\varphi_{\min}, \varphi_{\max}]$:** Aborta con `HTTP 422 Unprocessable Entity` (`PRICE_COLLAR_VIOLATION`) informando piso y techo exactos en `details` .
- **6a. Superación del límite diario de reventas por usuario:** Aborta con `HTTP 429 Too Many Requests` (`DAILY_RESALE_LIMIT_REACHED`).

UC-03: Ejecución Atómica de Compra Secundaria y Liquidación de Comisiones (Split P2P)

Atributo Normativo	Especificación Formal
Identificador y Endpoint	`UC-03` — `POST /api/v1/p2p/orders/:orderId/buy` (`p2pService.buyOrder`)
Actor Primario	Comprador Secundario autenticado (`buyerId`)
Stakeholders e Intereses	**Comprador:** Adquirir titularidad exclusiva e inmediata sin riesgo de doble cobro ni reutilización del QR por el vendedor anterior. **Vendedor:** Cobrar `netSellerProceeds` . **Comercio y Plataforma:** Registrar sus respectivas fracciones de comisión (`tenantNetFee` y `platformShare`) manteniendo `amount_in_escrow = 0.00` de variación sobre el principal.
Precondiciones	Existe una orden `orderId` en estado `OPEN` dentro del *Tenant* activo.
Garantía Mínima (En Fallo)	Si ocurre colisión concurrente o fallo de validación, ni la titularidad del cupón ni el libro mayor sufren mutación; exactamente a lo sumo un comprador puede triunfar.
Garantía de Éxito (Postcondición)	Orden pasa a `MATCHED` ; `coupons.current_owner_id = buyerId` , `state = 'EN_WALLET'` y `token_version = token_version + 1` ; asiento `P2P_SPLIT` (`amount_in_escrow = 0.00` , `tenant_fee_accumulated = tenantNetFee` , `platform_fee = platformShare`) insertado en `escrow_ledgers` .

Escenario Principal de Éxito (`UC-03`)

1. El Comprador selecciona una oferta abierta en el libro de órdenes del recinto y solicita su compra inmediata.
2. `p2pService` intenta adquirir `locks.withLock("order:" + orderId, ttlMs)`.
3. Con el candado adquirido, inicia `BEGIN ISOLATION LEVEL SERIALIZABLE` y bloquea la orden (`SELECT ... FOR UPDATE`) y su cupón asociado.
4. Verifica que `order.status === 'OPEN'`, que `buyerId !== order.sellerId` (prohibición de auto-negociación / *wash trading*) y que el mercado no haya entrado en ventana de cierre pre-evento.
5. Calcula con aritmética decimal exacta `calculateCommissionBreakdown` obteniendo `sellerFeeAmount`, `buyerFeeAmount`, `netSellerProceeds`, `totalBuyerPrice`, `platformShare` y `tenantNetFee`.
6. Actualiza `p2p_orders` a `status = 'MATCHED'` persistiendo `buyer_id`, `matched_at` y las cuatro columnas monetarias del cierre.
7. Transfiere el cupón a `current_owner_id = buyerId`, estado `EN_WALLET` e incrementa `token_version + 1`.
8. Inserta en `escrow_ledgers` el asiento `P2P_SPLIT` con `amount_in_escrow = 0.00` (el principal nominal original permanece inalterado en custodia), `tenant_fee_accumulated = tenantNetFee` y `platform_fee = platformShare`.
9. Registra eventos `P2P_SOLD` (para el vendedor) y `P2P_BOUGHT` (para el comprador), ejecuta `COMMIT`, libera el candado propietario en Redis/memoria, despacha ambos eventos por SSE y responde `HTTP 200 OK`.

Extensiones y Flujos Alternativos (`UC-03`)

- **2a. Orden bloqueada por otra compra concurrente en vuelo:** `LockManager` rechaza sin abrir transacción SQL devolviendo `HTTP 423 Locked` (`P2P_ORDER_LOCKED`).
- **4a. La orden ya fue adjudicada o cancelada milisegundos antes:** Aborta con `HTTP 409 Conflict` (`ORDER_NOT_OPEN`).
- **4b. Intento de autocompra (`buyerId === sellerId`):** Aborta con `HTTP 422 Unprocessable Entity` (`SELF_TRADE_FORBIDDEN`).
- **6a. Colisión de serialización en PostgreSQL (`SQLSTATE 40001`):** El manejador traduce el conflicto a rechazo controlado `HTTP 409/423` preservando atomicidad total (verificado con 50 compradores simultáneos en `QA-02`).

UC-04: Validación Criptográfica de QR Dinámico y Liquidación de Custodia en TPV

Atributo Normativo	Especificación Formal
Identificador y Endpoint	`UC-04` — `POST /api/v1/tpv/validate-qr` (`tpvService.validateAndRedeem`)
Actor Primario	Operador / Lector TPV del Comercio Emisor
Precondiciones	El consumidor exhibe un token QR generado por `GET /api/v1/coupons/:id/qr-token` dentro de los últimos 30 segundos.
Garantía Mínima (En Fallo)	Ante firma inválida, token vencido, recinto cruzado o versión obsoleta, el cupón no cambia de estado y **cero centavos** abandonan la cuenta de custodia.
Garantía de Éxito (Postcondición)	Cupón pasa al estado terminal `CANJEADO` (`token_version + 1`); se inserta asiento único `REDEEM_RELEASE` por `-nominalPrice` en `escrow_ledgers` (`uq_ledger_terminal_once`), cerrando en `0,00` el saldo retenido de ese cupón.

Escenario Principal de Éxito (`UC-04`)

1. El Lector TPV escanea el código QR rotativo de la pantalla del cliente y envía la cadena `token` al endpoint del *Tenant* del local.
2. `tpvService` decodifica el payload `base64url` y ejecuta `verifyQrToken(token, tenant.qrSecret, clock.now(), 30_000)` :

- Recalcula `HMAC-SHA256` con el `QrSecret` del *Tenant* receptor y verifica igualdad en tiempo constante (`timingSafeEqual`).
- Comprueba que `payload.tenantId === tenant.tenantId` .
- Comprueba vigencia temporal $|t_{actual} - t_{token}| \leq 30.000$ ms.

3. Adquiere candado sobre `"coupon:" + payload.couponId` y abre transacción `SERIALIZABLE` bloqueando el cupón (`FOR UPDATE`).
4. Verifica concordancia triple contra el estado persistido:

- `coupon.state === 'EN_WALLET'` (no está publicado en P2P, ni ya canjeado, ni vencido).
- `coupon.currentOwnerId === payload.ownerId` .
- `coupon.tokenVersion === payload.version` (defensa anti-boomerang).

5. Ejecuta la transición irreversible `EN_WALLET → CANJEADO` incrementando `token_version + 1` .
6. Inserta en `escrow_ledgers` el asiento `REDEEM_RELEASE` con `amount_in_escrow = -coupon.nominalPrice` .

7. Registra el evento `COUPON_REDEEMED`, confirma `COMMIT`, notifica por SSE a la billetera del consumidor y retorna `HTTP 200 OK` indicando `releasedToMerchant = nominalPrice`.

Extensiones y Flujos Alternativos (`UC-04`)

- **2a. Firma alterada o QR perteneciente a otro comercio (*Tenant* cruzado):** Rechazo inmediato `HTTP 401 Unauthorized` (`QR_INVALID_SIGNATURE` / `QR_TENANT_MISMATCH` — verificado en `QA-04`).
- **2b. Captura de pantalla con antigüedad superior a 30 segundos:** Rechazo `HTTP 401 Unauthorized` (`QR_TOKEN_EXPIRED`).
- **4a. Captura reciente (< 30 s) pero el cupón fue publicado o revendido tras emitirla (`payload.version` ≠ `coupon.tokenVersion`):** Rechazo `HTTP 409 Conflict` (`QR_STALE_TOKEN`).
- **4b. Intento de doble escaneo sobre cupón ya redimido:** Rechazo `HTTP 409 Conflict` (`COUPON_ALREADY_REDEEMED` + salvaguarda física `uq_ledger_terminal_once`).